



**UNIVERSIDAD TECNOLÓGICA
DE QUERÉTARO**

**INGENIERÍA EN DESARROLLO Y GESTIÓN DE
SOFTWARE**

GRUPO: IDGS14

EDUARDO MARTINEZ DUARTE

PROFESOR

Filiberto Ruiz Hernández

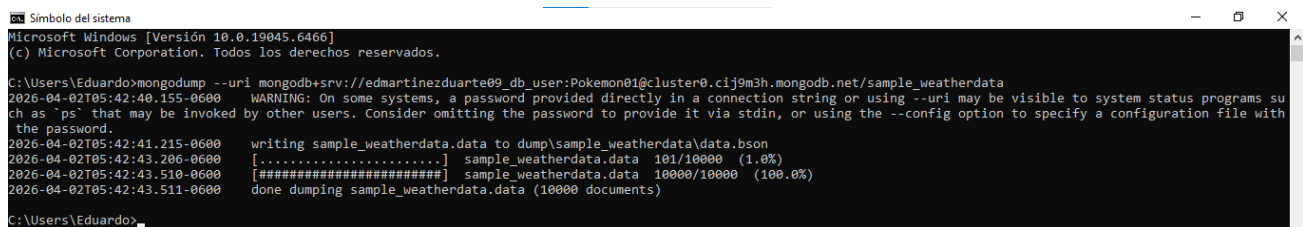
ACTIVIDAD

2a. evaluación

- **Procedimiento de copias de seguridad**

Para realizar la copia de seguridad de la base de datos no relacional se utilizó la herramienta mongodump, la cual forma parte de las MongoDB Database Tools y permite exportar los datos de una base de datos MongoDB en formato BSON.

Al ejecutar el comando desde la terminal, MongoDB comenzó a copiar todos los documentos de la base de datos sample_weatherdata. En la pantalla se pudo ver el avance del proceso hasta llegar al **100%**, lo que indica que los **10,000 documentos** fueron respaldados correctamente.

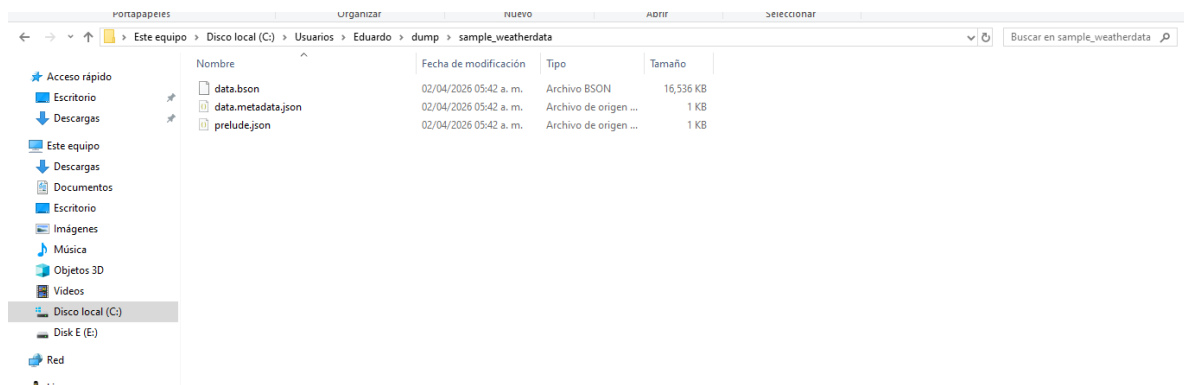


```
Símbolo del sistema
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Eduardo>mongodump --uri mongodb+srv://edmartinezduarte09_db_user:Pokemon01@cluster0.cij9m3h.mongodb.net/sample_weatherdata
2026-04-02T05:42:40.155-0600 WARNING: On some systems, a password provided directly in a connection string or using --uri may be visible to system status programs such as 'ps' that may be invoked by other users. Consider omitting the password to provide it via stdin, or using the --config option to specify a configuration file with the password.
2026-04-02T05:42:41.215-0600 writing sample_weatherdata.data to dump\sample_weatherdata\data.bson
2026-04-02T05:42:43.206-0600 [.....] sample_weatherdata.data 101/10000 (1.0%)
2026-04-02T05:42:43.510-0600 [#####] sample_weatherdata.data 10000/10000 (100.0%)
2026-04-02T05:42:43.511-0600 done dumping sample_weatherdata.data (10000 documents)

C:\Users\Eduardo>
```

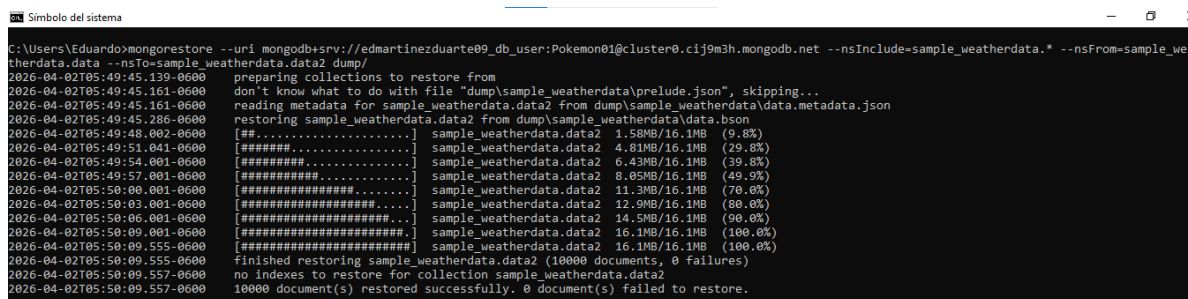
Una vez terminado el proceso, se generaron archivos de respaldo que quedaron guardados automáticamente en una carpeta llamada dump dentro del equipo, en la ruta C:\Users\Eduardo\dump\sample_weatherdata. Dentro de esa carpeta se pueden ver los archivos creados, entre ellos el archivo principal **data.bson** que contiene toda la información respaldada.



- **Protocolos de restauración**

Para restaurar la copia de seguridad que se había guardado anteriormente, se utilizó el comando **mongorestore**, que es la herramienta complementaria de mongodump y sirve para volver a cargar la información respaldada dentro de la base de datos.

Al ejecutar el comando, MongoDB comenzó a leer los archivos de la carpeta dump y fue cargando los documentos uno a uno, mostrando el avance en pantalla hasta completar el proceso al **100%**. Al finalizar, la terminal confirmó que los **10,000 documentos fueron restaurados exitosamente** y que no hubo ningún error durante el proceso.



```
C:\Users\Eduardo>mongorestore --uri mongodb+srv://edmartinezduarte09_db_user:Pokemon01@cluster0.cij9m3h.mongodb.net --nsInclude=sample_weatherdata.* --nsFrom=sample_weatherdata.data --nsTo=sample_weatherdata.data2 dump/
2026-04-02T05:49:45.139-0600 preparing collections to restore from
2026-04-02T05:49:45.161-0600 don't know what to do with file "dump/sample_weatherdata\prelude.json", skipping...
2026-04-02T05:49:45.161-0600 reading metadata for sample_weatherdata.data2 from dump/sample_weatherdata\data.metadata.json
2026-04-02T05:49:45.285-0600 restoring sample_weatherdata.data2 from dump/sample_weatherdata\data.bson
2026-04-02T05:49:48.002-0600 [##.....] sample_weatherdata.data2 1.58MB/16.1MB (9.8%)
2026-04-02T05:49:51.041-0600 [#####] sample_weatherdata.data2 4.81MB/16.1MB (29.8%)
2026-04-02T05:49:54.001-0600 [#####] sample_weatherdata.data2 6.43MB/16.1MB (39.8%)
2026-04-02T05:49:57.001-0600 [#####] sample_weatherdata.data2 8.05MB/16.1MB (49.9%)
2026-04-02T05:50:00.001-0600 [#####] sample_weatherdata.data2 11.3MB/16.1MB (70.0%)
2026-04-02T05:50:03.001-0600 [#####] sample_weatherdata.data2 12.9MB/16.1MB (80.0%)
2026-04-02T05:50:06.001-0600 [#####] sample_weatherdata.data2 14.5MB/16.1MB (90.0%)
2026-04-02T05:50:09.001-0600 [#####] sample_weatherdata.data2 16.1MB/16.1MB (100.0%)
2026-04-02T05:50:09.555-0600 [#####] sample_weatherdata.data2 16.1MB/16.1MB (100.0%)
2026-04-02T05:50:09.555-0600 finished restoring sample_weatherdata.data2 (10000 documents, 0 failures)
2026-04-02T05:50:09.557-0600 no indexes to restore for collection sample_weatherdata.data2
2026-04-02T05:50:09.557-0600 10000 document(s) restored successfully. 0 document(s) failed to restore.
```

Para verificar que la restauración funcionó correctamente, se abrió **MongoDB Compass**, que es la interfaz visual de MongoDB, y se pudo comprobar que dentro de la base de datos sample_weatherdata ahora aparecen dos colecciones:

- **data** — la colección original con 10,000 documentos.
- **data2** — la colección restaurada, también con 10,000 documentos y el mismo tamaño que la original.

Top Screenshot: MongoDB Shell

```

> use sample_weatherdata
< already on db sample_weatherdata
> show collections
< data
  data2
Atlas atlas-1193nu-shard-0 [primary] sample_weatherdata>
  
```

Bottom Screenshot: MongoDB Compass

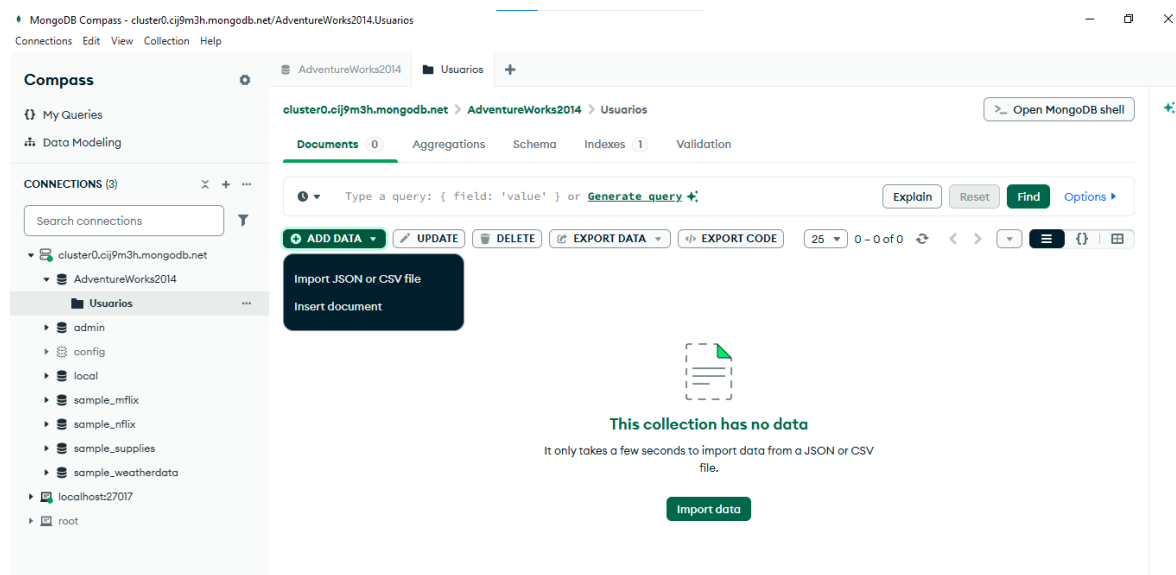
Database: sample_weatherdata

Collection name	Properties	Storage size	Data size	Documents	Avg. Document size	Indexes	Total Index size
data	-	4.80 MB	16.93 MB	10K	1.69 kB	1	303.10 kB
data2	-	2.42 MB	16.93 MB	10K	1.69 kB	1	303.10 kB

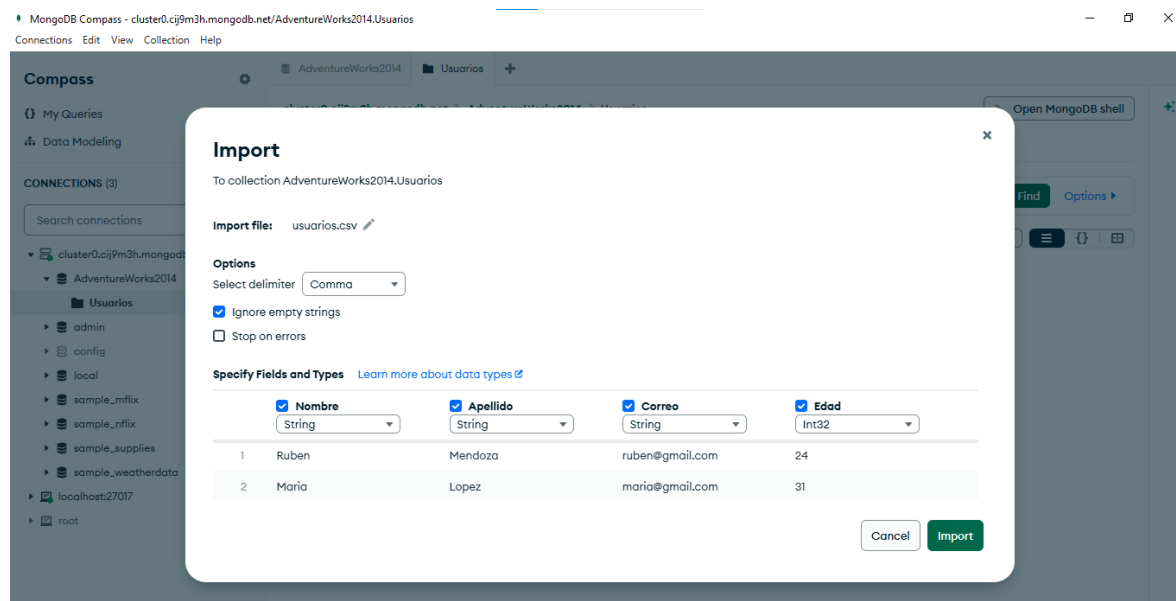
- **Métodos de importación y exportación (reutilice alguno de los archivos usados en la evaluación de bases de datos relacionales)**

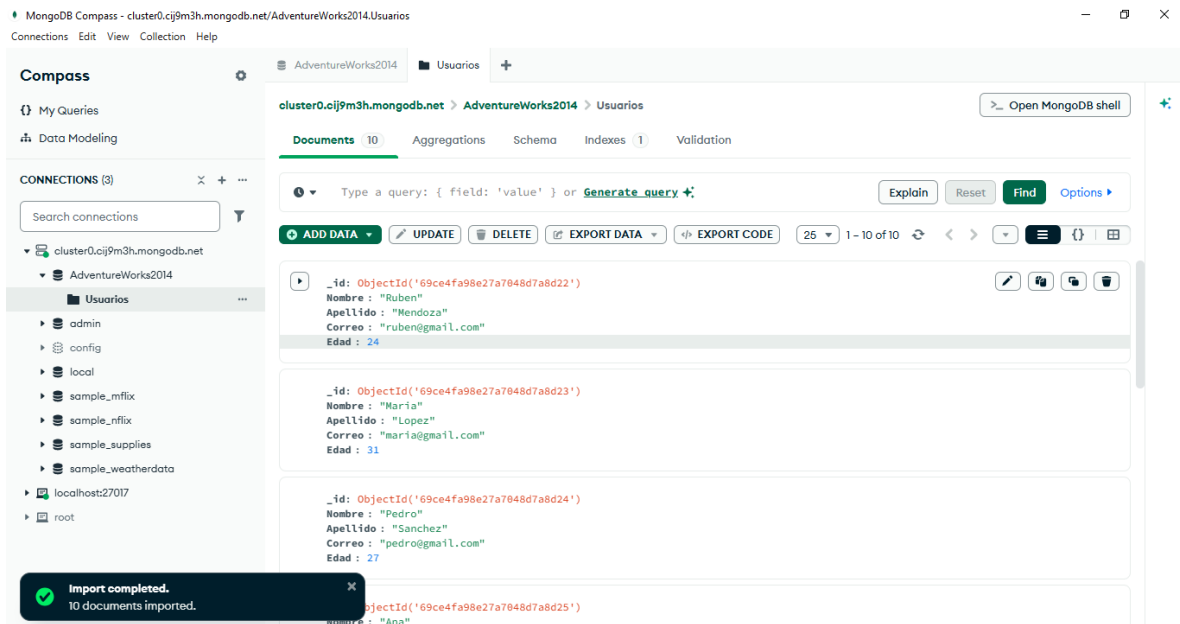
Para la parte de importación se creó una colección denominada **Usuarios** dentro de la base de datos **AdventureWorks2014**. Esta tabla fue diseñada para almacenar datos básicos como nombre, apellido, correo electrónico y edad

Primero se abrió la colección en MongoDB Compass, que al inicio aparecía vacía sin ningún documento. Para cargar los datos se utilizó la opción **Import JSON or CSV file** que ofrece la herramienta, y se seleccionó un archivo CSV que contenía los registros de varios usuarios.

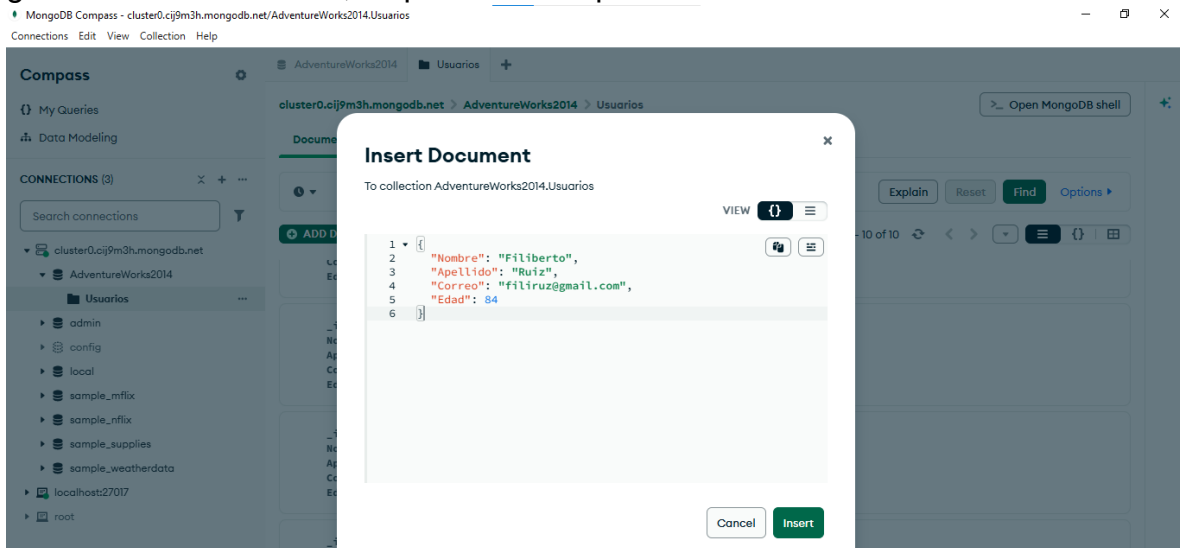


Una vez completada la importación, la pantalla confirmó el mensaje **"Import completed. 10 documents imported"**, lo que indica que los 10 registros del archivo fueron cargados correctamente en la colección. Al revisar los documentos, se puede ver que cada uno contiene la información correspondiente a cada usuario.

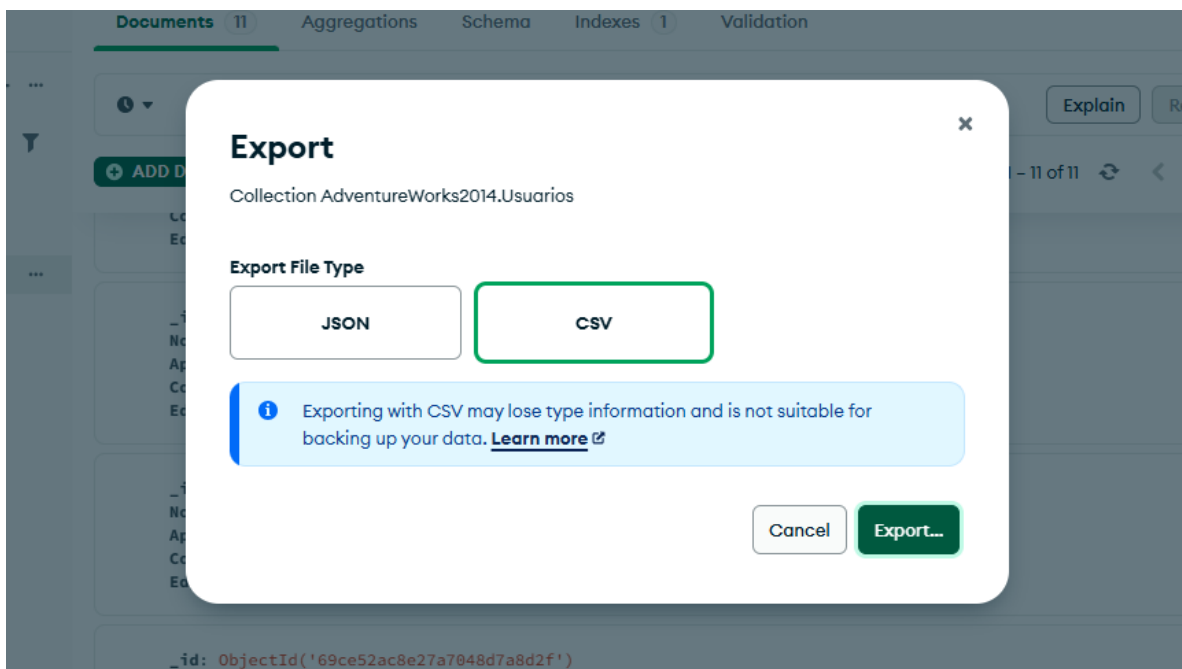
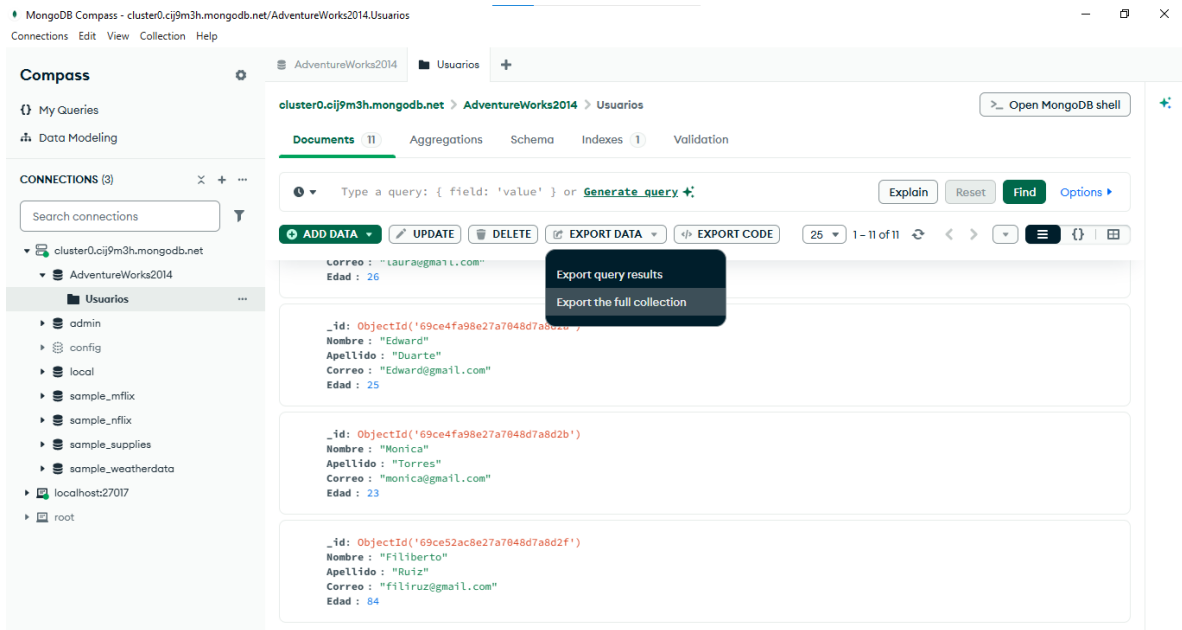




Para la parte de exportación, se insertó un nuevo usuario dentro de la colección Usuarios utilizando la opción **Insert Document** de MongoDB Compass. Una vez guardado el documento, se procedió a exportar toda la colección.



Para exportar, se utilizó la opción **Export Data** disponible en la barra de herramientas de Compass, donde se eligió el formato **CSV** como tipo de archivo de salida. Al abrir el archivo exportado en Excel, se puede ver que todos los usuarios de la colección quedaron organizados correctamente en columnas con sus respectivos campos: id, nombre, apellido, correo y edad, confirmando que la exportación se realizó de forma exitosa.




```

C:\Users\Eduardo>mongosh --port 27017
Current Mongosh Log ID: 69d0b4b6dd9c8512a2cb0ce1
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.3.8
Using MongoDB:      8.0.4
Using Mongosh:       2.3.8
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
  2026-04-04T00:33:10.917-06:00: Access control is not enabled for the database. Read and write access to data and configurati
  2026-04-04T00:33:10.919-06:00: This server is bound to localhost. Remote systems will be unable to connect to this server. S
to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior
_ip 127.0.0.1 to disable this warning
-----

test> use admin
switched to db admin
admin> db.createUser({
...   user: "admin",
...   pwd: "Pokemon01",
...   roles: [ { role: "userAdminAnyDatabase", db: "admin" }, "readWriteAnyDatabase" ]
... })
{ ok: 1 }
admin>

```

```

admin> use sample_weatherdata
switched to db sample_weatherdata
sample_weatherdata> db.createUser({
...   user: "usuario1_Edward",
...   pwd: "Pokemon01",
...   roles: [ { role: "readWrite", db: "sample_weatherdata" } ]
... })
{ ok: 1 }
sample_weatherdata>

```

```

sample_weatherdata> db.createUser({
...   user: "usuario2_Eduardo",
...   pwd: "Pokemon01",
...   roles: [ { role: "read", db: "sample_weatherdata" } ]
... })
{ ok: 1 }
sample_weatherdata>

```

Una vez creados ambos usuarios, se ejecutó el **comando db.getUsers()** para verificar que los dos quedaron registrados correctamente en la base de datos con sus respectivos roles.

```

sample_weatherdata> db.getUsers()
{
  users: [
    {
      _id: 'sample_weatherdata.usuario1_Edward',
      userId: UUID('46f6263c-2340-461f-a7ca-18d5844c2e08'),
      user: 'usuario1_Edward',
      db: 'sample_weatherdata',
      roles: [ { role: 'readWrite', db: 'sample_weatherdata' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'sample_weatherdata.usuario2_Eduardo',
      userId: UUID('5d7b12fa-eb93-4886-a01f-5ca763bec8d6'),
      user: 'usuario2_Eduardo',
      db: 'sample_weatherdata',
      roles: [ { role: 'read', db: 'sample_weatherdata' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
sample_weatherdata>

```

Con **usuario1_Edward** se pudo insertar un documento de prueba con el texto "Examen unidad 2" sin ningún problema, y también se pudo consultar dicho documento con el comando **db.data.find()**, confirmando que tiene acceso completo de lectura y escritura.

```

C:\Users\Eduardo>mongosh --port 27017 -u "usuario1_Edward" --authenticationDatabase "sample_weatherdata" -p
Enter password: *****
Current Mongosh Log ID: 69d0b8e22c223113f7cb0ce1
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=sample_weatherdata
Using MongoDB:      8.0.4
Using Mongosh:       2.3.8
Mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

```

```

sample_weatherdata> db.data.insertOne({ prueba: "Examen unidad 2" })
{
  acknowledged: true,
  insertedId: ObjectId('69d0b9462c223113f7cb0ce2')
}

```

```

sample_weatherdata> db.data.find({ prueba: "Examen unidad 2" })
[
  {
    _id: ObjectId('69d0b9462c223113f7cb0ce2'),
    prueba: 'Examen unidad 2'
  }
]
sample_weatherdata>

```

Con **usuario2_Eduardo**, al intentar insertar un documento, el sistema respondió con el mensaje **"not authorized on sample_weatherdata to execute command insert"**, lo que confirma que este usuario solo tiene permiso de lectura y no puede realizar escrituras.

```
mongosh mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=sample_weatherdata

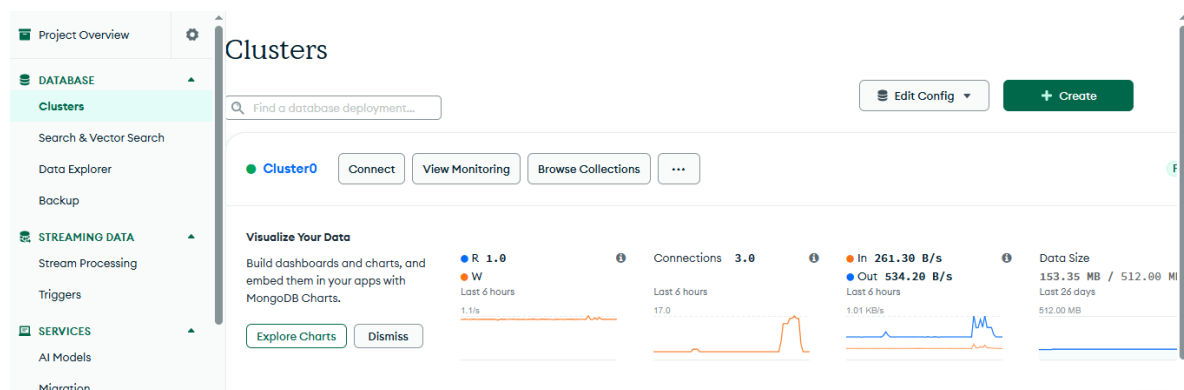
C:\Users\Eduardo>mongosh --port 27017 -u "usuario2_Eduardo" --authenticationDatabase "sample_weatherdata" -p
Enter password: *****
Current Mongosh Log ID: 69d0ba6ebf557c1df8cb0ce1
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&authSource=sample_weatherdata
Using MongoDB:      8.0.4
Using Mongosh:       2.3.8
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
```

```
sample_weatherdata> db.data.findOne()
{
  _id: ObjectId('69d0b9462c223113f7cb0ce2'),
  prueba: 'Examen unidad 2'
}
sample_weatherdata> db.data.insertOne({ prueba: "sin permiso" })
MongoServerError[Unauthorized]: not authorized on sample_weatherdata to execute command { insert: "data", documents: [ { prueba: "sin permiso" } ], ordered: true, lsid: { id: UUID("f4dcd45f-6dc7-4454-b9f9-1d9c8c84d080") }, $db: "sample_weatherdata" }
sample_weatherdata>
```

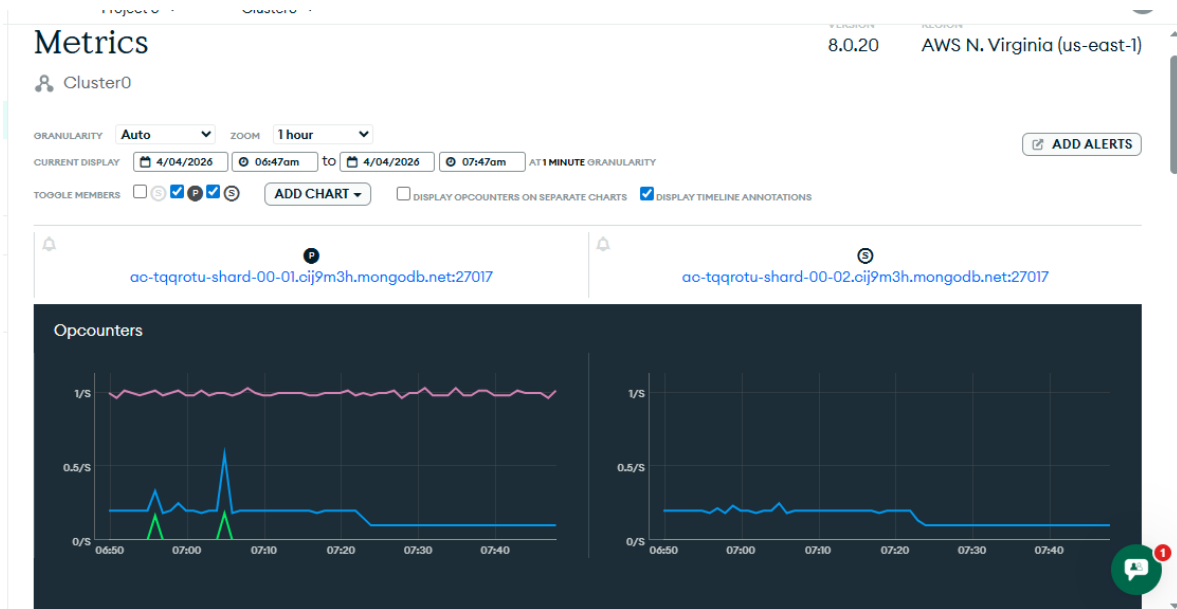
Elabore un reporte que contenga:

- El rendimiento de la base de datos no relacional a partir de los datos que arroja las herramientas de monitoreo (en tiempo real y en la nube)

El panel principal de MongoDB Atlas muestra un resumen del estado del clúster **Cluster0** en tiempo real. En la sección de métricas se pueden observar los siguientes datos durante las últimas 6 horas



1. **Operaciones:** Se registró un pico de actividad de **1.0 operaciones**, lo que corresponde al momento en que se realizaron las pruebas de inserción y consulta desde la terminal.
2. **Conexiones:** Se mantuvieron **3.8 conexiones** activas en promedio, que corresponden a las sesiones abiertas desde mongosh y MongoDB Compass durante las pruebas.
3. **Tráfico de red:** Se registró un tráfico de entrada de **261.38 B/s** y de salida de **534.20 B/s**, lo que refleja el intercambio de datos entre el cliente y el servidor durante las operaciones realizadas.
4. **Tamaño de la base de datos:** El espacio utilizado es de **153.35 MB de un total disponible de 512 MB**, lo que indica que la base de datos tiene suficiente capacidad disponible y no presenta problemas de almacenamiento.





En general, las métricas muestran que el clúster funcionó de manera estable durante todas las pruebas, sin alertas ni problemas de rendimiento.

Para poder usar las herramientas de monitoreo, primero se crearon dos roles especiales llamados **mongostatRole** y **mongotopRole**, y se asignaron al usuario1_Edward junto con su rol de escritura.

```

C:\> mongosh mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&server
test> use admin
switched to db admin
admin> db.createRole(
...   {
...     role: "mongostatRole",
...     privileges: [
...       { resource: { cluster: true },
...         actions: [ "serverStatus" ] }
...     ],
...     roles: []
...   }
... )
{ ok: 1 }
admin> use admin
already on db admin
admin> db.createRole(
...   {
...     role: "mongotopRole",
...     privileges: [
...       { resource: { cluster: true },
...         actions: [ "top" ] }
...     ],
...     roles: []
...   }
... )
{ ok: 1 }

```

Asignar ambos roles a un usuario

```
admin> use sample_weatherdata
switched to db sample_weatherdata
sample_weatherdata> db.updateUser("usuario1_Edward", {
...   roles: [
...     { role: "mongostatRole", db: "admin" },
...     { role: "mongotopRole", db: "admin" },
...     { role: "readWrite", db: "sample_weatherdata" }
...   ]
... })
{ ok: 1 }
sample_weatherdata> _
```

Mongostat (estadísticas en tiempo real)

Con la herramienta **mongostat** se pudo observar en tiempo real las operaciones que estaba ejecutando el servidor cada segundo, mostrando información como inserciones, consultas, actualizaciones y el uso de memoria.

```
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Eduardo>mongostat --port 27017 -u usuario1_Edward -p Pokemon01 --authenticationDatabase sample_weatherdata
2026-04-04T01:56:45.046-0600 WARNING: On some systems, a password provided directly using --password may be visible to system status programs such as 'ps' that may
be invoked by other users. Consider omitting the password to provide it via stdin, or using the --config option to specify a configuration file with the password.
insert query update delete getmore command dirty used flushes vsize res qrw arw net in net out conn time
*0 *0 *0 *0 0 0 0 0.0% 0.0% 0 5.15G 751M 0 0 0 110b 82.6k 8 Apr 4 01:56:47.129
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 114b 85.4k 8 Apr 4 01:56:48.108
*0 *0 *0 *0 0 0 0 0.0% 0.0% 0 5.15G 751M 0 0 0 109b 82.2k 8 Apr 4 01:56:49.127
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 245b 84.0k 8 Apr 4 01:56:50.127
*0 *0 *0 *0 0 2 0 0.0% 0.0% 0 5.15G 751M 0 0 0 166b 85.2k 8 Apr 4 01:56:51.113
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 112b 83.9k 8 Apr 4 01:56:52.111
*0 *0 *0 *0 0 0 0 0.0% 0.0% 0 5.15G 751M 0 0 0 111b 83.0k 8 Apr 4 01:56:53.120
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 112b 84.1k 8 Apr 4 01:56:54.115
*0 *0 *0 *0 0 2 0 0.0% 0.0% 0 5.15G 751M 0 0 0 316b 82.9k 8 Apr 4 01:56:55.132
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 112b 84.2k 8 Apr 4 01:56:56.126
insert query update delete getmore command dirty used flushes vsize res qrw arw net in net out conn time
*0 *0 *0 *0 0 0 0 0.0% 0.0% 0 5.15G 751M 0 0 0 111b 83.3k 8 Apr 4 01:56:57.131
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 112b 84.2k 8 Apr 4 01:56:58.125
*0 *0 *0 *0 0 0 0 0.0% 0.0% 0 5.15G 751M 0 0 0 111b 83.4k 8 Apr 4 01:56:59.129
*0 *0 *0 *0 0 2 0 0.0% 0.0% 0 5.15G 751M 0 0 0 249b 85.3k 8 Apr 4 01:57:00.114
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 162b 83.2k 8 Apr 4 01:57:01.124
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 112b 84.3k 8 Apr 4 01:57:02.116
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 112b 84.0k 8 Apr 4 01:57:03.112
*0 *0 *0 *0 0 1 0 0.0% 0.0% 0 5.15G 751M 0 0 0 112b 84.0k 8 Apr 4 01:57:04.109
```

Mongotop (tiempo por colección)

Con la herramienta **mongotop** se visualizó cuánto tiempo dedicaba MongoDB a cada colección para operaciones de lectura y escritura.

```
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Eduardo>mongotop --port 27017 -u usuario1_Edward -p Pokemon01 --authenticationDatabase sample_weatherdata
2026-04-04T01:57:37.025-0600 WARNING: On some systems, a password provided directly using --password may be visible to system status p
e invoked by other users. Consider omitting the password to provide it via stdin, or using the --config option to specify a configuration
2026-04-04T01:57:37.076-0600 connected to: mongod://localhost:27017/

ns      total  read  write  2026-04-04T01:57:38-06:00
admin.atlascli      0ms    0ms    0ms
admin.system.roles  0ms    0ms    0ms
admin.system.users  0ms    0ms    0ms
admin.system.version 0ms    0ms    0ms
config.system.sessions 0ms    0ms    0ms
config.transactions 0ms    0ms    0ms
local.system.replset 0ms    0ms    0ms
sample_weatherdata.data 0ms    0ms    0ms

ns      total  read  write  2026-04-04T01:57:39-06:00
admin.atlascli      0ms    0ms    0ms
admin.system.roles  0ms    0ms    0ms
admin.system.users  0ms    0ms    0ms
admin.system.version 0ms    0ms    0ms
config.system.sessions 0ms    0ms    0ms
config.transactions 0ms    0ms    0ms
local.system.replset 0ms    0ms    0ms
sample_weatherdata.data 0ms    0ms    0ms

ns      total  read  write  2026-04-04T01:57:40-06:00
admin.atlascli      0ms    0ms    0ms
admin.system.roles  0ms    0ms    0ms
admin.system.users  0ms    0ms    0ms
admin.system.version 0ms    0ms    0ms
config.system.sessions 0ms    0ms    0ms
config.transactions 0ms    0ms    0ms
local.system.replset 0ms    0ms    0ms
sample_weatherdata.data 0ms    0ms    0ms
```

Al ejecutar el comando **db.serverStatus()** podemos obtener la información sobre el estado del servidor en nuestra consola y en nuestra base de datos.

```
admin> db.serverStatus()
{
  host: 'Edward',
  version: '8.0.4',
  process: 'mongod',
  service: [ 'shard' ],
  pid: Long('10016'),
  uptime: 1495,
  uptimeMillis: Long('1486901'),
  uptimeEstimate: Long('1486'),
  localTime: ISODate('2026-04-04T08:02:33.254Z'),
  asserts: {
    regular: 0,
    warning: 0,
    msg: 0,
    user: 66,
    tripwire: 0,
    rollovers: 0
  },
  batchedDeletes: {
    batches: 2,
    docs: 4,
    stagedSizeBytes: 972,
    timeInBatchMillis: 0,
    refetchesDueToyield: 0
  },
  catalogStats: {
    collections: 20,
    capped: 0,
    clustered: 0,
    timeseries: 0,
    views: 0,
  }
}
```

“Gestión de la calidad de los datos”

Caso de estudio --- Salud Vital

Introducción

SaludVital es una cadena de clínicas médicas que enfrenta serios problemas con la calidad de sus datos: registros de pacientes repetidos, formatos inconsistentes, información incompleta en expedientes clínicos, errores en facturación y dificultades para rastrear el inventario de insumos médicos. Cada uno de estos problemas tiene consecuencias reales: un paciente con dos expedientes puede recibir un tratamiento incorrecto; una factura con datos erróneos genera rechazos de aseguradoras; un inventario desactualizado puede dejar sin insumos a una clínica en plena jornada.

Este documento presenta una propuesta estructurada para mejorar la calidad de los datos gestionados en MongoDB, la base de datos no relacional utilizada por SaludVital. La propuesta aborda once dimensiones clave: exactitud, integridad, disponibilidad, confiabilidad, utilidad, accesibilidad, pertinencia, usabilidad, actualización, normalización y eliminación de duplicados. Cada dimensión se analiza con sus causas, riesgos y acciones concretas de mejora.

1. Exactitud — Datos que reflejan la realidad

Un dato exacto es aquel que corresponde fielmente al estado real de la información. En una clínica médica, la exactitud es crítica: una dosis mal registrada, un diagnóstico incorrecto o una fecha de nacimiento errónea puede provocar daños al paciente. En SaludVital, la exactitud se ve comprometida cuando el personal captura información sin validación.

1.1 Validación automática de entradas

MongoDB permite definir reglas de validación dentro de cada colección usando JSON Schema. Con esta herramienta se puede establecer que el campo "correo" siga el formato de una dirección de correo electrónico válida, que "fechaNacimiento" sea una fecha real y que "peso" esté dentro de un rango fisiológico aceptable. Cualquier documento que no cumpla las reglas será rechazado antes de guardarse.

1.2 Verificación con fuentes oficiales

Cuando se registra un paciente nuevo, el sistema debería verificar que el nombre y número de identificación coincidan con la información oficial. Un simple error tipográfico en el nombre puede generar dos expedientes distintos para la misma persona, fragmentando su historial médico.

1.3 Revisión periódica de datos críticos

Es necesario establecer un calendario de auditorías donde el personal clínico revise datos sospechosos: pesos o edades fuera de rango, campos con valores genéricos como "N/A" o "desconocido", y registros sin información médica básica. Corregir un dato a tiempo evita decisiones incorrectas en el futuro.

2. Integridad — Datos completos y coherentes

La integridad implica que la información esté completa, que no falten datos importantes, y que los registros relacionados entre sí sean coherentes. En MongoDB, a diferencia de las bases relacionales, no existen llaves foráneas automáticas, por lo que la integridad debe cuidarse desde el diseño y la lógica de la aplicación.

2.1 Campos obligatorios en el esquema

Mediante la validación de esquema se pueden marcar como requeridos los campos esenciales de cada colección. Por ejemplo, ningún documento de paciente debería guardarse sin nombre completo, número de identificación, tipo de sangre y alergias conocidas. Si alguno de estos campos falta, MongoDB rechaza el documento y solicita que se complete.

2.2 Consistencia entre colecciones relacionadas

Cuando se registra una consulta médica, deben crearse simultáneamente el registro clínico y el registro de facturación. Si solo se crea uno de los dos, se genera una inconsistencia que dificulta el cobro y el seguimiento. MongoDB permite usar transacciones para que ambas operaciones ocurran juntas o ninguna de las dos.

2.3 Completitud activa

No basta con requerir campos al momento de la captura. Es necesario ejecutar consultas periódicas que detecten registros con campos vacíos o con valores nulos y notifiquen al equipo responsable para que los complete. MongoDB permite hacer estas verificaciones con pipelines de agregación.

3. Disponibilidad — Datos accesibles cuando se necesitan

En un entorno clínico con múltiples sucursales, la base de datos debe estar disponible las 24 horas del día. Una caída del sistema durante una emergencia médica puede tener consecuencias graves. MongoDB ofrece herramientas robustas para garantizar alta disponibilidad.

3.1 Replicación del clúster

Con un conjunto de réplicas (replica set), MongoDB mantiene copias sincronizadas de los datos en varios servidores. Si el servidor principal falla, uno de los

secundarios toma su lugar de forma automática en segundos, sin intervención manual y sin pérdida de información.

3.2 Respaldos programados

MongoDB Atlas ofrece la función de copias de seguridad automáticas con puntos de restauración. Esto permite recuperar la base de datos al estado que tenía en cualquier momento de los últimos días, protegiéndola ante borrados accidentales o corrupción de datos.

3.3 Escalabilidad ante alta demanda

Si el volumen de datos crece significativamente —por ejemplo, al integrar más clínicas— se puede aplicar sharding, que distribuye las colecciones entre varios servidores para repartir la carga. Esto garantiza que el sistema responda rápido incluso con millones de registros.

4. Confiabilidad — Datos en los que se puede confiar

Un sistema confiable es aquel en el que médicos y administradores pueden basar sus decisiones sin dudar de la veracidad de la información. La confiabilidad es el resultado de aplicar correctamente todas las demás dimensiones de calidad.

4.1 Auditoría de cambios

En MongoDB Atlas se puede habilitar el registro de auditoría, que guarda un historial de quién modificó qué dato y en qué momento. Esto permite rastrear el origen de cualquier cambio y detectar modificaciones no autorizadas o errores introducidos por el personal.

4.2 Control de acceso por roles

Solo el personal autorizado debe poder modificar información sensible. Un médico puede actualizar diagnósticos, pero no debería poder modificar registros de facturación. El sistema de roles de MongoDB permite definir estos permisos con precisión, reduciendo el riesgo de cambios accidentales o malintencionados.

4.3 Verificaciones de consistencia periódicas

A través del aggregation framework de MongoDB se pueden ejecutar consultas de verificación: ¿los totales de las facturas coinciden con los pagos registrados? ¿El inventario refleja correctamente las entradas y salidas de insumos? Estas comprobaciones deben programarse de forma regular para detectar discrepancias a tiempo.

5. Utilidad — Datos que sirven para algo concreto

De nada sirve una base de datos grande si la información almacenada no se utiliza para mejorar la atención o la operación. La utilidad mide si los datos realmente ayudan a tomar decisiones, generar reportes y resolver problemas del día a día en SaludVital.

5.1 Identificar los datos más valiosos

Se debe definir cuáles son los datos de mayor impacto: historial clínico completo (antecedentes, alergias, medicamentos activos), datos de contacto actualizados, información del seguro médico y niveles de inventario. Estos campos deben priorizarse en el diseño de la base y en los procesos de captura.

5.2 Estructurar pensando en las consultas más frecuentes

MongoDB permite anidar información relacionada dentro de un mismo documento. Por ejemplo, las visitas recientes de un paciente pueden guardarse dentro de su propio documento, evitando búsquedas costosas en colecciones separadas. Este diseño hace que las consultas más comunes sean más rápidas y sencillas.

5.3 Eliminar datos que ya no aportan valor

Registros de pacientes que nunca asistieron, datos de prueba que quedaron en producción, o campos que nadie consulta son ejemplos de información que ocupa espacio y dificulta los análisis. Limpiar periódicamente la base mejora su utilidad y su rendimiento.

6. Accesibilidad — Datos fáciles de encontrar y usar

La accesibilidad no es solo que el sistema esté en línea, sino que cualquier persona autorizada pueda encontrar la información que necesita en segundos, sin conocimientos técnicos avanzados.

6.1 Interfaces amigables para el personal

Herramientas como MongoDB Compass permiten consultar la base de datos de forma visual sin necesidad de escribir código. Para el personal de recepción o enfermería, lo ideal es contar con una aplicación interna con formularios de búsqueda simples: buscar paciente por nombre, ver citas del día, consultar stock de un insumo.

6.2 Índices en campos de búsqueda frecuente

Crear índices en campos como el número de identificación del paciente, su apellido o el código de un medicamento hace que las consultas sean casi instantáneas, incluso con millones de registros. Sin índices, MongoDB recorre toda la colección para encontrar un resultado, lo que puede tardar varios segundos.

6.3 Base de datos centralizada en la nube

Al alojar la base en MongoDB Atlas, todas las sucursales de SaludVital acceden a la misma información en tiempo real. Esto elimina el problema de que cada clínica maneje su propia base local con datos diferentes y desincronizados. Al mismo tiempo, los permisos garantizan que cada persona solo vea lo que le corresponde.

7. Pertinencia — Solo los datos que realmente importan

La pertinencia significa que cada dato almacenado tiene un propósito claro dentro de la organización. Guardar información innecesaria no solo desperdicia espacio: complica los análisis, dificulta las búsquedas y puede generar problemas de privacidad.

7.1 Cada campo debe justificarse

Antes de agregar un nuevo campo a la base, el equipo debe preguntarse: ¿para qué lo vamos a usar? Las alergias a medicamentos son altamente pertinentes; los pasatiempos del paciente, no. En el documento de inventario, el nivel de stock mínimo es pertinente; el color del empaque, probablemente no.

7.2 Evitar la acumulación de datos sin uso

MongoDB, al ser flexible, permite agregar campos libremente. Esto puede convertirse en un problema si no hay disciplina: documentos con decenas de campos de los cuales solo cinco se consultan. Un esquema bien definido y revisado periódicamente evita esta acumulación.

7.3 Revisar la pertinencia conforme cambia el negocio

Con el tiempo, las necesidades de la clínica cambian. Lo que hoy es pertinente puede dejar de serlo mañana, y viceversa. Por ejemplo, el estado de vacunación del paciente puede volverse relevante ante una nueva normativa sanitaria. MongoDB facilita agregar campos sin necesidad de migraciones complejas, lo que permite adaptarse rápidamente.

8. Usabilidad — Datos fáciles de leer y entender

Un dato usable es aquel que cualquier persona autorizada puede interpretar correctamente sin ambigüedad. Nombres de campos crípticos, formatos inconsistentes o estructuras demasiado complejas reducen la usabilidad y generan errores de interpretación.

8.1 Nombres de campos claros y consistentes

En lugar de abreviaturas como "fNac" o "tel1", usar nombres descriptivos como "fechaNacimiento" o "telefonoPrincipal". Esta práctica beneficia tanto al personal

técnico que desarrolla la aplicación como al personal de salud que eventualmente revisa los datos directamente.

8.2 Diccionario de datos

Documentar el significado de cada campo, sus valores posibles y sus unidades de medida. Por ejemplo, indicar que "temperatura" se registra en grados Celsius, o que "estadoPaciente" puede tener los valores "activo", "inactivo" o "fallecido". Este diccionario evita interpretaciones incorrectas y facilita la incorporación de nuevo personal.

8.3 Formatos homogéneos en toda la base

Todas las fechas deben guardarse en el mismo formato (ISODate en UTC), los números telefónicos con el mismo patrón, y las direcciones separadas en campos individuales (calle, número, colonia, código postal). La homogeneidad facilita los reportes y las comparaciones entre registros.

9. Actualización — Datos vigentes en todo momento

Un dato desactualizado puede ser tan peligroso como un dato incorrecto. Si el expediente de un paciente muestra un medicamento que ya no toma, o si el inventario indica stock disponible cuando en realidad está agotado, las consecuencias pueden ser graves. La información debe reflejar siempre el estado actual de las cosas.

9.1 Actualizar en el momento del evento

Cada vez que ocurre algo relevante —una consulta, una compra de insumos, un pago— el registro correspondiente debe actualizarse de inmediato. MongoDB ofrece operadores atómicos como \$set e \$inc que permiten modificar un campo específico sin riesgo de sobreescribir otros datos.

9.2 Tareas automáticas para actualizaciones periódicas

Algunos datos necesitan actualizarse de forma regular aunque no ocurra un evento específico. Por ejemplo, marcar automáticamente como "inactivo" a cualquier paciente que no haya tenido una consulta en los últimos 12 meses. MongoDB Atlas permite configurar triggers y tareas programadas para este tipo de actualizaciones.

9.3 Historial de cambios cuando sea necesario

En algunos casos, actualizar no significa borrar el valor anterior. Si un paciente cambia de domicilio o de seguro médico, conviene guardar el dato antiguo con su fecha de vigencia y registrar el nuevo como activo. Esto mantiene la trazabilidad sin perder el estado actual.

10. Normalización — Datos con formato uniforme

En el contexto de MongoDB, normalización no se refiere a las formas normales del modelo relacional, sino a la estandarización de formatos y convenciones. SaludVital tiene registros donde el mismo paciente aparece como "Juan Pérez", "juan perez", "JUAN PÉREZ" o "Pérez, Juan". Esta variación hace que los sistemas no reconozcan que se trata de la misma persona.

10.1 Definir estándares antes de capturar

Establecer desde el inicio que todos los nombres se registran con mayúscula inicial, que las fechas usan formato ISO en UTC, que los números de teléfono incluyan el código de país y siguen el patrón +52-XXX-XXXXXXX. Las interfaces de captura deben aplicar estas reglas automáticamente.

10.2 Limpiar los datos históricos

Para los datos ya almacenados con formatos inconsistentes, se puede usar el aggregation framework de MongoDB para detectar y corregir variantes masivamente. Un pipeline puede identificar todas las variantes de un campo y convertirlas al formato estándar en una sola operación.

10.3 Cuidado con la redundancia

En MongoDB es común guardar copias de un dato en varios documentos para evitar consultas costosas. Esta práctica es válida, pero requiere disciplina: si el nombre de un paciente se guarda también dentro de sus citas, cualquier cambio en el nombre debe propagarse a todos los documentos que lo contienen. De lo contrario, se crean versiones contradictorias del mismo dato.

11. Eliminar Duplicados — Un registro por entidad

Los registros duplicados de pacientes fueron uno de los primeros problemas identificados en SaludVital. Un paciente con dos expedientes puede recibir atención basada en información incompleta, lo que representa un riesgo médico. En administración, los duplicados generan facturas repetidas, confusión en pagos y reportes incorrectos.

11.1 Detectar duplicados existentes

Con el aggregation framework de MongoDB se puede agrupar los documentos por campos clave como CURP, número de expediente o combinación de nombre y fecha de nacimiento. Los grupos con más de un documento representan posibles duplicados que deben revisarse y fusionarse.

11.2 Índices únicos para prevenir futuros duplicados

Crear un índice único en el campo de identificación del paciente garantiza que MongoDB rechace automáticamente cualquier intento de insertar un documento con un identificador ya existente. Esta restricción actúa como una red de seguridad que evita duplicados desde el momento de la captura.

11.3 Procedimiento operativo en recepción

La tecnología por sí sola no es suficiente. El personal de recepción debe tener el hábito de buscar al paciente en el sistema antes de crear un registro nuevo. La interfaz debe mostrar una advertencia clara si detecta una posible coincidencia con un paciente existente, basándose en nombre, fecha de nacimiento o CURP.

Resultados esperados

La aplicación sistemática de estas once estrategias transformará la base de datos de SaludVital en un sistema confiable y eficiente. Cada paciente estará representado por un único expediente completo y actualizado. Las facturas serán exactas y estarán alineadas con los registros clínicos. El inventario reflejará en tiempo real el estado de cada insumo.

El personal médico podrá tomar decisiones con la seguridad de que la información es correcta. El personal administrativo podrá generar reportes sin tener que limpiar datos antes de analizarlos. Y los directivos podrán confiar en los indicadores que el sistema produce, porque estarán basados en datos de calidad.

Conclusión

La calidad de los datos no es un proyecto de una sola vez: es un proceso continuo que requiere reglas claras, herramientas adecuadas y compromiso del equipo. MongoDB ofrece todas las capacidades técnicas necesarias para implementar las estrategias descritas en este documento: validación de esquemas, índices únicos, transacciones, agregaciones, replicación y auditoría.

Sin embargo, ninguna herramienta sustituye a las buenas prácticas del equipo humano. La combinación de tecnología bien configurada y procesos bien definidos es lo que convierte una base de datos en un activo estratégico para SaludVital. Invertir en la calidad de los datos es invertir en la calidad de la atención médica y en la sostenibilidad del negocio.